Design and Implementation of a simple 7-Stage Pipelined RISC-V Processor

Introduction

This article presents the development of a 7-stage pipelined RISC-V processor. We outline the architectural evolution from a basic single-cycle core to a full RV32I-compliant processor. By transitioning to a 7-stage pipeline, we significantly optimized the system's instruction throughput. Finally, the design was integrated with a UART peripheral and deployed on an FPGA, where it successfully executed C-based applications, proving its reliability in a real hardware environment.

Single-Cycle Processor

Design

This microarchitecture executes instructions in a single cycle. Figure 1.1 shows a basic structure of a single-cycle processor.

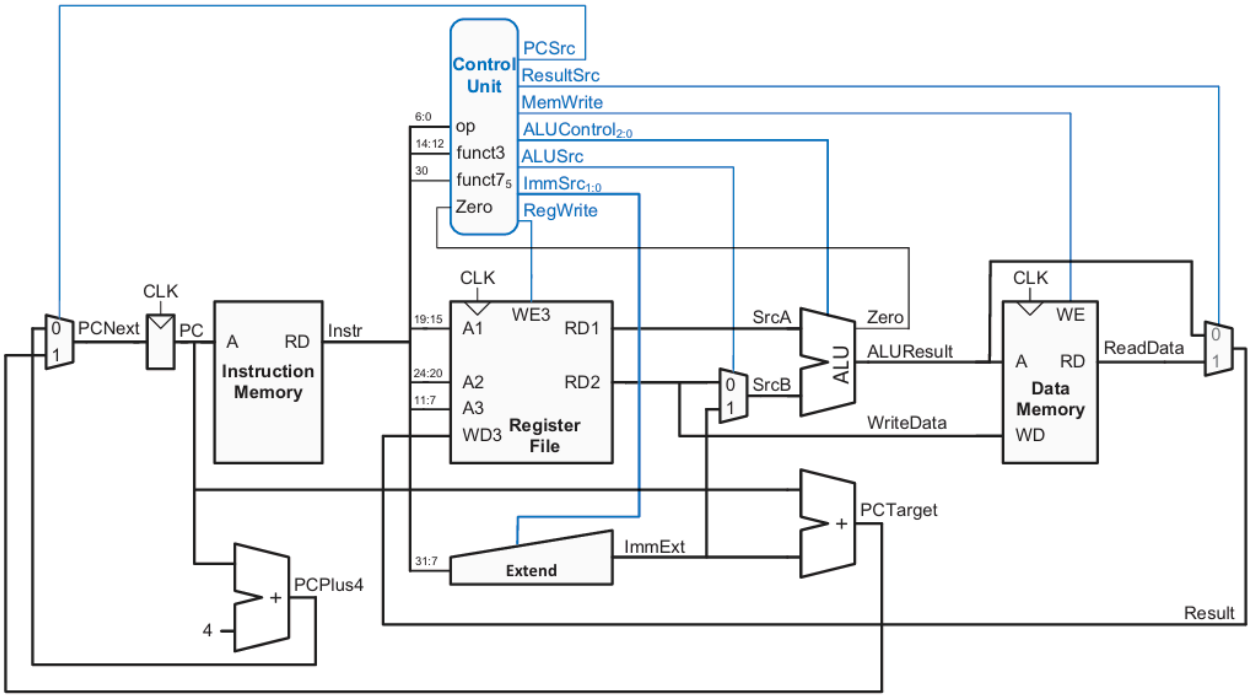

Figure 1.1: Basic Single-Cycle Processor

However, this structure supports only a few RV32I instructions, changes must be made to handle every RV32I instruction.

1. Control Flow and Jump Support (JAL, JALR)

The basic structure in Figure 1.1 typically handles simple branches (B-Type), but full RV32I compliance requires unconditional jumps (J-Type and I-Type). To support `jal` (Jump And Link) and `jalr` (Jump And Link Register), we modified the Next PC logic and the Write-Back logic:

- **Next PC Selection:** The `jalr` instruction requires jumping to an address calculated by the ALU (register + immediate), rather than the PC-relative target used by branches. We expanded the `PCNext` multiplexer logic to select between `PC + 4`, `PCTarget` (branch/jal), and `ALUResult` (jalr).

```
always @( *) begin
    case (pc_src)
        2'b00: pc_next = pc_plus4;
        2'b01: pc_next = pc_target;
        2'b10: pc_next = alu_result;
        default: pc_next = pc_plus4;
    endcase
end
```

- **Return Address Storage:** Both `jal` and `jalr` need to store the return address (`PC + 4`) into the register file. We added a result selection multiplexer (`ResultSrc`) to allow writing `PC + 4` directly to the destination register, alongside `ALUResult` and `ReadData`.

```
always @( *) begin
    case (result_src)
        2'b00: result = alu_result;
        2'b01: result = mem_data_processed;
        2'b10: result = pc_plus4;
        default: result = alu_result;
    endcase
end
```

2. Upper Immediate Instructions (LUI, AUIPC)

To support `lui` (Load Upper Immediate) and `auipc` (Add Upper Immediate to PC), we modified the ALU input multiplexers:

- **ALU Source A:** Standard instructions use the register file output `RD1`. However, `auipc` requires the current `PC` as an operand. For `lui`, the controller selects `0` for Source A and the immediate value for Source B, effectively passing the immediate through the ALU. We implemented a multiplexer for `SrcA` to select between `RD1`, `PC`, or `0`.

```
always @( *) begin
    case (alu_src_a)
        2'b00: src_a = rf_rd1;
        2'b01: src_a = pc;
        2'b10: src_a = 32'b0;
        default: src_a = rf_rd1;
    endcase
end
```

3. Sub-Word Memory Access (Load/Store)

The basic architecture assumes 32-bit word alignment. To fully support RV32I, we implemented logic for byte and half-word operations (`lb`, `lh`, `lbu`, `lhu`, `sb`, `sh`):

- **Load Logic:** We introduced a post-processing block for the data memory output. Based on the `funct3` field and the byte offset (from `ALUResult`), this logic handles sign-extension or zero-extension for bytes and half-words before writing to the register file.

```
// take lb, lbu as an example
always @( *) begin
    case (funct3)
        // lb
        3'b000: begin
            case (byte_offset)
                2'b00: mem_data_processed = {{24{read_data[7]}}, read_data[7:0]};
                2'b01: mem_data_processed = {{24{read_data[15]}}, read_data[15:8]};
                2'b10: mem_data_processed = {{24{read_data[23]}}, read_data[23:16]};
                2'b11: mem_data_processed = {{24{read_data[31]}}, read_data[31:24]};
            endcase
        end

        // lbu
        3'b100: begin
            case (byte_offset)
                2'b00: mem_data_processed = {24'b0, read_data[7:0]};
                2'b01: mem_data_processed = {24'b0, read_data[15:8]};
                2'b10: mem_data_processed = {24'b0, read_data[23:16]};
                2'b11: mem_data_processed = {24'b0, read_data[31:24]};
            endcase
        end
    endcase
end
```

- **Store Logic:** The Data Memory (`dmem`) module was updated to handle write-enable signals for specific byte lanes, allowing the processor to modify only 8 or 16 bits of a 32-bit word in memory.

```
always @(posedge clk) begin
    if (we) begin
        case (funct3)
            // sw
            3'b010: RAM[addr[11:2]] <= wd;
            // sh
            3'b001: begin
                if (addr[1] == 0)   // low
                    RAM[addr[11:2]][15:0] <= wd[15:0];
                else
                    RAM[addr[11:2]][31:16] <= wd[15:0];
            end
            // sb
            3'b000: begin
                case (addr[1:0])
```

```
                    2'b00: RAM[addr[11:2]][7:0]     <= wd[7:0];
                    2'b01: RAM[addr[11:2]][15:8]    <= wd[7:0];
                    2'b10: RAM[addr[11:2]][23:16]   <= wd[7:0];
                    2'b11: RAM[addr[11:2]][31:24]   <= wd[7:0];
                endcase
            end
            default: RAM[addr[11:2]] <= wd;
        endcase
    end
end
```

4. ALU Functionality Expansion

The basic ALU was upgraded to handle the complete set of arithmetic and logical operations required by RV32I:

- **Comparison:** We added logic for both signed (`slt`, `blt`, `bge`) and unsigned (`sltu`, `bltu`, `bgeu`) comparisons.
- **Shifts:** The ALU was expanded to support logical shifts (`sll`, `srl`) and arithmetic right shifts (`sra`).

Operation	alu_control	ex. instr
ADD	4'b0000	add, addi, lw, sw, jal
SUB	4'b0001	sub, beq, bne
AND	4'b0010	and, andi
OR	4'b0011	or, ori
XOR	4'b0100	xor, xori
SLT	4'b0101	slt, slti
SLTU	4'b0110	sltu, sltiu
SLL	4'b0111	sll, slli
SRL	4'b1000	srl, srli
SRA	4'b1001	sra, srai

5. Control Unit Integration

Finally, the control unit was implemented to decode the 7-bit opcode, `funct3`, and `funct7` fields. It generates the expanded control signals—such as `ImmSrc`, `ALUSrcA`, `ALUSrcB`, and `PCSrc`—to coordinate the datapath modifications described above.

Verification

For the Single-Cycle Processor, we implemented a preliminary verification framework to ensure the correctness of the core architecture before moving to the more complex pipelined design. The verification process consists of two parts: automated co-simulation for architectural compliance and a C-based on-board test for peripheral interaction.

> Note: A more comprehensive verification suite covering data hazards and advanced control flows will be detailed in the Pipelined Processor section of this report.

1. Automated Co-Simulation with Cocotb

To verify the execution of standard RV32I instructions, we developed an automated testbench using Cocotb. This environment compares the state of the RTL design against a Python-based Golden Model.

- **Reference Model:**
  We built a CPU model that behaves as an instruction-set simulator. It parses the assembly code and calculates the expected state of the Register File and Data Memory for every instruction.
- **Assembly Test Case:**
  A custom assembly program (`main.asm`) was written to cover various instruction types, including R-Type arithmetic, I-Type immediate operations, Load/Store memory access, and Branch/Jump control flows.
- **Result Checking:**
  The testbench executes the assembly code on the RTL and, upon completion, dumps the values of all 32 general-purpose registers and the relevant data memory regions. These values are automatically compared against the Golden Model's output.
    - Regfile and Dmem will be checked like this:

```
# Check regs
rf_handle = dut.dut.d_unit.rf.regs
mismatch =  False
for i in range(0, 32):
    rtl_val = int(rf_handle[i].value)
    exp_val = expected_regs[i]
    if rtl_val != exp_val:
        cocotb.log.error(f"Mismatch at x{i}! Expected: {hex(exp_val)}, Got:
{hex(rtl_val)}")
        mismatch = True
if not mismatch:
    cocotb.log.info("PASS: All registers match model!")
else:
    raise Exception("FAIL: Register mismatch detected!")
```

    - If test passed, cocotb will return the result:

```
500000.00ns INFO     cocotb.regression      test_top.verify passed
500000.00ns INFO     cocotb.regression
**************************************************************************************************
** TEST                          STATUS  SIM TIME (ns)  REAL TIME (s)  RATIO (ns/s) **
**************************************************************************************************
** test_top.verify                PASS      500000.00           0.14     3467857.26  **
**************************************************************************************************
** TESTS=1 PASS=1 FAIL=0 SKIP=0             500000.00           0.15     3437807.57  **

**************************************************************************************************
```

2. FPGA On-Board Verification

In addition to simulation, we validated the processor's capability to execute high-level compiled code on actual hardware. We wrote a simple C language program to control the LEDs on the FPGA board.

The program (`main.c`) utilizes Memory-Mapped I/O (MMIO) to toggle an LED connected to a specific address (0x80000000). It implements a delay loop to create a blinking effect. This test verifies the entire toolchain—from C compilation to instruction fetching and MMIO store operations. The processor successfully executed the program, resulting in the LED blinking as expected on the FPGA.

Pipelined Processor

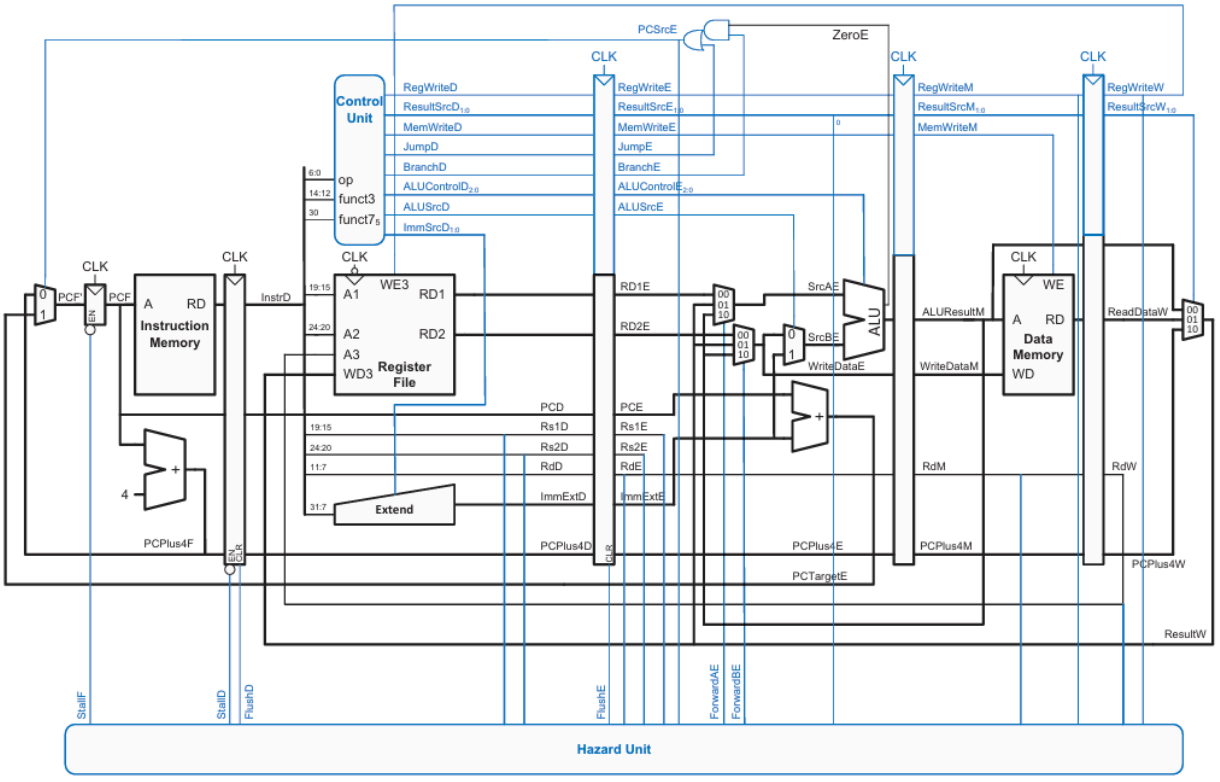

Figure 2.1: Pipelined processor with full hazard handling

Architecture Overview

To achieve higher operating frequencies and better compatibility with FPGA hardware resources (specifically Block RAMs), we evolved the single-cycle architecture into a 7-stage pipelined processor. Standard 5-stage pipelines assume asynchronous memory access (data available in the same cycle), but modern FPGA memory blocks are synchronous, requiring at least one clock cycle to fetch data.

Our 7-stage design splits the Instruction Fetch (IF) and Memory Access (MEM) stages to accommodate this latency without reducing the clock speed. The stages are defined as follows:

- **IF1 (Fetch 1):**
  The Program Counter (PC) is updated, and the address is sent to Instruction Memory.
- **IF2 (Fetch 2):**
  The instruction data is received from Memory and latched into the pipeline register.
- **ID (Decode):**
  Instruction decoding, register file reading, and immediate extension.
- **EX (Execute):**
  ALU operations and branch resolution.
- **M1 (Memory 1):**
  The ALU result is used as an address for Data Memory access.
- **M2 (Memory 2):**
  Read data is received from Data Memory and processed (byte/half-word alignment).
- **WB (Writeback):**
  Results are written back to the Register File or CSRs.

Hazard Management

Pipelining introduces dependencies between instructions that must be resolved to ensure correct execution. We implemented a dedicated Hazard Unit (`hazard_unit.v`) to handle Data Hazards, Control Hazards, and Structural/Load-Use Hazards.

1. Data Hazards and Forwarding

Since the Writeback stage (WB) is several cycles after the Execute stage (EX), an instruction needing a result from a previous instruction would typically have to wait. To minimize stalls, we implemented Forwarding logic.

The Hazard Unit monitors the source registers (Rs1, Rs2) in the Decode/Execute stages and compares them with the destination registers (Rd) in the M1, M2, and WB stages. If a match is found (and RegWrite is enabled), the latest data is forwarded directly to the ALU inputs.

Forwarding Paths

Data can be forwarded to the EX stage from:

- **M1 Stage:** The ALU result of the immediately preceding instruction.
- **M2 Stage:** The result from two cycles ago.
- **WB Stage:** The result from three cycles ago.

```
// take SrcB as an example
```

```
// hazard_unit.v
always @( *) begin
    if (((Rs2_E_H == Rd_M1_H) && RegWrite_M1_H) && (Rs2_E_H != 0))
        ForwardB_E_r = 2'b10;
    else if (((Rs2_E_H == Rd_M2_H) && RegWrite_M2_H) && (Rs2_E_H != 0))
        ForwardB_E_r = 2'b11;
    else if (((Rs2_E_H == Rd_W_H) && RegWrite_W_H) && (Rs2_E_H != 0))
        ForwardB_E_r = 2'b01;
    else
        ForwardB_E_r = 2'b00;
    end
// datapath.v
always @( *) begin
    case (ForwardB_E)
        2'b00: WriteData_E = RD2_E;
        2'b01: WriteData_E = Result_W;
        2'b10: WriteData_E = Result_M1;
        2'b11: WriteData_E = Result_M2;
        default: WriteData_E = RD2_E;
    endcase
end
assign SrcB_E = (ALUSrc_b_E) ? ImmExt_E : WriteData_E;
```

2. Load-Use Hazards and Multi-Cycle Stalls

Unlike ALU operations where the result is available immediately after the Execute stage, memory load instructions (lw, lb, etc.) incur significant latency. In our 7-stage architecture, data from memory is not available until the end of the M2 stage. This creates a hazard window of three cycles where Forwarding is impossible because the data has not yet arrived from the memory subsystem.

To handle this, we implemented a robust stall logic in the Hazard Unit that monitors the three pipeline stages ahead of the Decode stage.

Hazard Detection Logic

The logic identifies a hazard by checking if a Load instruction exists in the EX, M1, or M2 stages and if its destination register (Rd) matches the source registers (Rs1/Rs2) of the instruction currently in the ID stage.

The detection code utilizes the LSB of the ResultSrc signal (`ResultSrc[0]`), which is set to 1 specifically for Load instructions (where ResultSrc = 2'b01 or 2'b11 for CSR):

```
always @( *) begin
    case (ResultSrc_W)
        2'b00: Result_W = ALU_Result_W;
        2'b01: Result_W = ReadData_Processed_W;
        2'b10: Result_W = PC_Plus4_W;
        2'b11: Result_W = CSR_ReadData;
        default: Result_W = ALU_Result_W;
    endcase
end
```

When a stall is asserted to freeze the fetch and decode stages (`Stall_F`, `Stall_D`), the pipeline must ensure that the Execute stage does not process invalid or duplicate instructions in the next cycle.

To achieve this, the lwStall signal also triggers Flush_E. This clears the ID/EX pipeline registers, effectively inserting a "Bubble" (NOP) into the Execute stage. This bubble propagates through the back-end stages (M1, M2, WB), ensuring no state changes occur while the dependent instruction waits in the Decode stage for the memory data to become available.

```
assign lwStall =
    // Case 1: Load in EX stage vs Instruction in ID
    (ResultSrc_E_0_H  && (Rd_E_H != 5'b0)  && ((Rs1_D_H == Rd_E_H) || (Rs2_D_H == Rd_E_H)))
||
    // Case 2: Load in M1 stage vs Instruction in ID
    (ResultSrc_M1_0_H && (Rd_M1_H != 5'b0) && ((Rs1_D_H == Rd_M1_H) || (Rs2_D_H ==
Rd_M1_H))) ||
    // Case 3: Load in M2 stage vs Instruction in ID
    (ResultSrc_M2_0_H && (Rd_M2_H != 5'b0) && ((Rs1_D_H == Rd_M2_H) || (Rs2_D_H ==
Rd_M2_H)));

assign Flush_E = (lwStall || (|PC_Src_E_H)) || EX_Flush_H;
```

3. Control Hazards (Branching)

Branch decisions are resolved in the EX stage. We employ a "Predict Not Taken" strategy. The processor continues fetching instructions sequentially (PC+4).

If the branch is not taken, execution continues normally.

If the branch is taken, the instructions currently in the F1, F2, and ID stages are invalid. The Hazard Unit asserts Flush signals (Flush_F2, Flush_D, Flush_E) to clear these pipeline registers, and the PC is updated to the correct branch target.

Exceptions and Interrupts

To support a realistic operating system environment and handle asynchronous events, we implemented a complete Machine-Mode CSR (Control and Status Register) unit defined in `csr_file.v`.

1. CSR Architecture

We implemented the essential Machine-Mode CSRs required for a basic RISC-V trap handler. These registers are mapped to their standard 12-bit addresses:

- **mstatus (0x300):** Machine Status Register.
  It tracks the global interrupt enable (MIE) bit and the previous interrupt enable (MPIE) bit.
- **mie (0x304) / mip (0x344):** Machine Interrupt Enable / Pending Registers.
  Used to mask and track interrupt sources (External, Timer, Software).
- **mtvec (0x305):** Machine Trap-Vector Base-Address.
  Stores the address where the PC jumps to when a trap occurs.
- **mepc (0x341):** Machine Exception PC.
  Stores the PC of the instruction that caused the exception (or the interrupted instruction).
- **mcause (0x342):** Machine Cause.
  Stores an ID indicating the reason for the trap (e.g. interrupt or illegal instruction).

- **mtval (0x343):** Machine Trap Value.
  Stores additional information (e.g. the faulting address in a Load/Store fault).

2. Interrupt Request Logic

The processor supports three types of interrupts: External (MEIP), Timer (MTIP), and Software (MSIP). The logic in csr_file.v continuously monitors external interrupt signals and updates the mip register:

```
mip[11] <= ext_int;     // MEIP
mip[7]  <= timer_int;   // MTIP
mip[3]  <= sw_int;      // MSIP
```

To decide if an interrupt should be serviced, the CSR unit performs a bitwise AND between pending interrupts (`mip`) and enabled interrupts (`mie`). The result is combined with the global interrupt enable (`mstatus[3]`, i.e. MIE) in the main controller to assert the interrupt_pending signal:

```
assign global_int_en = mstatus[3];      // mstatus.MIE
assign pending_interrupts = mip & mie;  // if != 1, interrupt exists
assign interrupt_pending = |pending_interrupts;
```

3. Trap Entry Mechanism

When the writeback stage asserts `trap_en` (indicating an exception or interrupt is taken), the CSR file automatically updates the architectural state to preserve context and prepare for the handler. This is critical for correct execution.

- **Save PC:**
  The current PC (or the next PC for interrupts) is saved to mepc.

  ```
  mepc <= trap_pc;
  ```

- **Record Cause:**
  The reason for the trap is written to mcause.

  ```
  mcause <= trap_cause;
  ```

- **Update Status (mstatus):**
  The processor must disable interrupts to prevent nested traps from overwriting mepc before software can save it. We implemented a hardware stack for the interrupt enable bit:
  - Save the current MIE (bit 3) to MPIE (bit 7).
  - Set MIE (bit 3) to 0 to globally disable interrupts.

  ```
  mstatus[7] <= mstatus[3];   // MPIE --> store old state
  mstatus[3] <= 1'b0;         // MIE  --> disbale interrupt
  ```

4. Trap Return Mechanism (MRET)

The `mret` instruction is used to return from a trap handler. When the `is_mret` signal is asserted, the CSR file reverses the operations performed during trap entry:

- **Restore Status:**
  The logic restores the global interrupt enable state from MPIE back to MIE.

  ```
  mstatus[3] <= mstatus[7];
  ```

- **Set MPIE to 1:**
  To avoid errors in Interrupt Nesting, MPIE is reset to 1.

  ```
  mstatus[7] <= 1'b1;
  ```

Bus Interface and AXI Adaptation

To enable communication between the high-speed processor pipeline and the system bus, we implemented an AXI-Lite Bridge. This module bridges the simple memory interface of the CPU (MemWrite, Addr, WData) with the handshake-based AXI4-Lite protocol.

1. AXI FSM Implementation

The bridge converts the processor's single-cycle memory requests into multi-cycle AXI4-Lite transactions using a Finite State Machine (FSM). The FSM ensures protocol compliance and manages the processor's stall signal (`cpu_stall`).

- **IDLE:**
  The FSM waits for a cpu_req. Upon a request, it immediately asserts the necessary AXI valid signals (`AWVALID`/`WVALID` for writes, `ARVALID` for reads) and transitions to the active state.
- **Write Path:** (`WR_ADDR_DATA`, `WR_RESP`)
  - **WR_ADDR_DATA:**
    To optimize throughput, the FSM attempts to handshake both the Write Address and Write Data channels simultaneously. It stays in this state until the slave accepts the data.
  - **WR_RESP:**
    Once the data is sent, the FSM waits for the Write Response (`BVALID`) from the slave to confirm transaction completion.
- **Read Path:** (`RD_ADDR`, `RD_DATA`)
  - **RD_ADDR:** Asserts the Read Address and waits for the slave's `ARREADY`.
  - **RD_DATA:** Waits for `RVALID`, latches the incoming `RDATA` into the `cpu_rdata` register, and terminates the read transaction.
- **WAIT_HANDSHAKE:** (Synchronization)
  - This is a critical termination state. The stall logic is defined as

    ```
    assign cpu_stall = cpu_req && (state != WAIT_HANDSHAKE);
    ```

    - By transitioning to `WAIT_HANDSHAKE` after a transaction completes, the bridge *de-asserts* `cpu_stall` for exactly one cycle, allowing the processor pipeline to advance and sample the valid `cpu_rdata` before the FSM returns to `IDLE`.

2. Address Stability Mechanism

Directly interfacing a pipelined processor with a multi-cycle bus introduces signal stability challenges. The AXI protocol requires the address and control signals to *remain stable* once `VALID` is asserted. However, the processor's pipeline registers in the Memory stage might drift if the pipeline is stalled improperly or if combinatorial logic fluctuates.

To resolve this, we implemented a dedicated address decoder register:

```
reg is_uart_addr_M2;    // Latches the target peripheral selection
always @(posedge clk_core or posedge reset) begin
    if (reset)
        is_uart_addr_M2 <= 1'b0;
    else if (!AXI_Stall)
        is_uart_addr_M2 <= is_uart_addr;
end
```

- **Function:**
  This register captures whether the current memory access targets the UART peripheral range at the beginning of the M2 stage.
- **Why it is needed:**
  During the multi-cycle AXI transaction (e.g. while the FSM is in RD_DATA waiting for the bus), the processor core is frozen. This register ensures that the data multiplexer (`ReadData`) consistently selects the AXI Bridge's output (`cpu_rdata`) throughout the entire stall duration, preventing data corruption or glitches before the valid data arrives.

  ```
  assign ReadData = is_uart_addr_M2 ? uart_rdata : bram_rdata;
  ```

Design Trade-offs and Performance Analysis

Transiting from a single-cycle to a 7-stage pipeline introduces a trade-off between clock frequency and Instruction Per Cycle (IPC).

- **Frequency Improvement:**
  By breaking down critical paths (especially the memory access paths into F1/F2 and M1/M2), the logic depth per stage is reduced, allowing for a significantly higher operating frequency on the FPGA.
- **IPC Cost:**
  While the ideal IPC is 1, hazards introduce penalties.
    - **Branch Misprediction:**
      A taken branch incurs a 3-cycle penalty (flushing F1, F2, ID).
    - **Load-Use Hazard:**
      A load instruction followed immediately by a dependent instruction incurs a latency stall.

    - **Memory Latency:**
      The 2-cycle memory access is hidden by the pipeline for independent instructions but contributes to the branch/load penalties.
- **Conclusion:**
  Despite the lower IPC compared to a single-cycle machine, the substantial increase in clock speed results in a net performance gain for typical workloads.

Verification

To ensure the reliability of the processor, we adopted a comprehensive verification strategy combining RTL simulation for architectural compliance and FPGA emulation for system-level integration.

1. RTL Simulation

We verified the core's adherence to the RISC-V RV32I standard using the official rv32ui-p test suite.

- **Methodology:**
  We developed a TCL script to automate the testing process. The script iterates through the test suite, loading the hex file for each instruction into the simulation memory.
- **Results:**
  The processor passed the majority of the tests, confirming correct integer arithmetic and control flow logic.
    - **Exceptions:**
      The ma_data (misaligned data) test failed as intended, because our design traps misaligned accesses as exceptions rather than handling them in hardware.

      ```
      wire Addr_Misaligned =
          ((Funct3_E == 3'b010) && (ALU_Result_E[1:0] != 0)) || // lw, sw  -->  4-aligned
          ((Funct3_E == 3'b001 || Funct3_E == 3'b101) && (ALU_Result_E[0] != 0)); // lh, lhu, sh --> 2-aligned
      ```

      Additionally, the fence.i test passed by treating the instruction as a NOP, which is compliant for our simple core without a cache hierarchy.

```
========================================
Test Summary
========================================
Total Passed: 41
Total Failed: 1
Failed Tests:
  - rv32ui-p-ma_data.hex
========================================
```

2. FPGA Emulation and Software Testing

Following simulation, we synthesized the design for FPGA to validate peripheral interaction and real-world execution.

- **Software Stack:**
  We wrote a custom startup file (`start.S`) and linker script (`link.ld`) to initialize the stack pointer and handle the C runtime environment.
- **UART & Polling:**
  The system successfully communicated via UART. For the demo applications, we utilized a *polling-based* driver. This design choice was made because the current UART hardware implementation shares interrupt logic for TX and RX without separation, and the interrupt latency made high-speed interrupt-driven I/O less stable for these specific demonstrations.

Demos

To strictly validate the processor's capability to execute high-level C code and interact with peripherals in a real-world environment, we developed and deployed three distinct demonstration programs. Each program targets specific aspects of the architecture, from basic I/O to complex data manipulation.

1. **Basic Connectivity:** Hello World & Echo (`demo.c`)
   This is the foundational test used to verify the reliability of the UART link and the system boot process.
   - **Functionality:**
     Upon system reset, the processor initializes the UART controller and transmits a "Hello World!" banner. It then enters an infinite loop, monitoring the RX FIFO. Any character received from the host PC is immediately transmitted back (echoed) to the terminal.
   - **Verification Goal:**
     This demo confirms that the instruction fetch path, the stack initialization, and the basic MMIO (Memory-Mapped I/O) read/write operations for the UART peripheral are functioning correctly. It serves as the "sanity check" for the entire system.
   - **Observed Result:**
     The terminal displays the greeting message, and keystrokes are responsive without data corruption, confirming a stable 9600 baud rate connection.

```
[Rx] Hello World!
[Tx] RISC-V
[Rx] RISC-V
[Tx] This is a 7-Stage Pipelined RISC-V Processor.
[Rx] This is a 7-Stage Pipelined RISC-V Processor.
```

2. **Arithmetic Stress Test:** Fibonacci Generator (`fibonacci_gen.c`)
   This program tests the processor's arithmetic logic unit (ALU) and register file stability over long periods of execution.
   - **Functionality:**
     The program continuously calculates the Fibonacci sequence ($F_n = F_{n-1} + F_{n-2}$). To make the output human-readable, a software delay loop is inserted between each iteration, slowing down the printing speed.
   - **Verification Goal:**
     This verifies the correctness of ADD instructions, register data forwarding (as dependencies are tight in the calculation), and the processor's ability to handle continuous operation without accumulation errors.

- **Observed Result:**
  The consistent delay demonstrates the processor's reliable execution of long instruction sequences and the correct handling of the NOP-based delay loop without unexpected pipeline stalls or drifts.

```
[Rx] === Fibonacci Generator ===
[Rx] 1
[Rx] 1
[Rx] 2
[Rx] 3
[Rx] 5
[Rx] 8
[Rx] 13
[Rx] 21
[Rx] 34
[Rx] 55
[Rx] 89
[Rx] 144
[Rx] 233
[Rx] 377
[Rx] 610
[Rx] 987
[Rx] 1597
[Rx] 2584
[Rx] 4181
[Rx] 6765
...
```

3. **Interactive Data Processing:** Bubble Sort (`bubble_sort.c`)
   This is the most complex demo, designed to test memory operations, nested control flow, and string parsing.
   - **Functionality:**
     The program implements an interactive command-line interface. It prompts the user to input a series of unsorted numbers (space-separated). The system reads the input string into a buffer, parses the characters into an integer array, sorts them using the Bubble Sort algorithm, and prints the result.
   - **Verification Goal:** This application comprehensively tests:
     - **Control Flow:**
       Nested for loops and if conditions used in sorting logic heavily stress the branch prediction and flushing mechanisms.
     - **Memory Access:**
       Frequent array reads/writes verify the Load/Store unit and the Data Memory interface.
     - **Logic:**
       String-to-Integer conversion tests complex arithmetic and bitwise operations.
   - **Observed Result:**
     The processor correctly parses arbitrary inputs and outputs the sorted sequence, demonstrating full functional equivalence to a standard computer execution.

```
[Rx] === Bubble Sort ===
[Rx] Enter numbers:
[Tx] 1 1 4 5 1 4
[Rx] 1 1 4 5 1 4
[Rx] Sorting 6 numbers...
[Rx] Result: 1 1 1 4 4 5
[Rx]
[Rx] Enter numbers:
[Tx] 2 0 2 5 5 3 1 0 6 4
[Rx] 2 0 2 5 5 3 1 0 6 4
[Rx] Sorting 10 numbers...
[Rx] Result: 0 0 1 2 2 3 4 5 5 6
```

Conclusion and Future Work

Conclusion

In this project, we successfully designed and implemented a fully functional (RV32I) 32-bit RISC-V processor from scratch. The development process evolved from a basic single-cycle architecture to a high-performance 7-stage pipelined core, addressing the critical challenge of synchronous Block RAM latency in FPGA environments.

Key achievements of this work include:

- **Architectural Optimization:**
  By splitting the Instruction Fetch and Memory Access stages, we resolved the timing constraints imposed by FPGA hardware, allowing for higher operating frequencies compared to standard 5-stage designs.
- **Robust Hazard Management:**
  The implementation of a comprehensive Hazard Unit effectively resolves data dependencies via forwarding and control hazards via flushing, ensuring correct execution logic.
- **System-Level Integration:**
  The processor was successfully integrated with an AXI-Lite bridge and UART peripherals. The verification results, ranging from RTL simulation to real-world FPGA demos (Fibonacci, Bubble Sort), demonstrate the core's capability to execute complex C programs reliably.

Future Work

While the current design meets all functional requirements, several avenues for optimization remain:

- **Dynamic Branch Prediction:**
  Currently, the processor uses a static "Predict-Not-Taken" strategy. Implementing a Branch History Table (BHT) or a Branch Target Buffer (BTB) would significantly reduce the flush penalty for taken branches, improving IPC (Instructions Per Cycle).
- **Cache Hierarchy:**
  To support larger memory spaces, future iterations could introduce Instruction and Data Caches to hide memory access latency.

- **Interrupt Controller Upgrade:**
  Transitioning from the current simple CSR-based interrupt handling to a PLIC (Platform-Level Interrupt Controller) would allow for prioritized handling of multiple peripheral interrupts.
- **DMA Support:**
  Implementing a Direct Memory Access (DMA) controller for the UART module would offload data transfer tasks from the CPU, preventing the high overhead observed in interrupt-driven I/O.

Appendix: FPGA Implementation Details

To validate the design on physical hardware, we synthesized and implemented the processor on the ZYNQ7020 using Xilinx Vivado. Key hardware primitives, including Clock Management and Memory Generators, were instantiated to ensure system stability and performance.

Clock Management (PLL)

- **IP Used:** Xilinx Clocking Wizard (MMCM).
- **Configuration:**
  - **Input Frequency:** 50..00 MHz (Source: System Clock)
  - **Output Frequency:** 60.00 MHz (Domain: `clk_core`)
- **Reset Logic:** The IP provides a locked signal. The system reset (`sys_rst_n`) is logically ANDed with this locked signal to ensure the processor remains in reset until the clock signal stabilizes.

Memory

Since our 7-stage pipeline can fetch an instruction (IF stage) and access data (MEM stage) in the same clock cycle, a single-port memory would cause structural hazards. To resolve this without complex arbitration logic, we utilized a True Dual-Port Block RAM.

- **IP Used:** Block Memory Generator (BRAM).
- **Configuration:**
  - **Memory Type:** True Dual Port RAM.
  - **Port A (Instruction Fetch):**
    Connected to the IF stage (PC Address). Read-Only mode.
  - **Port B (Data Access):**
    Connected to the MEM stage (ALU Result Address). Read/Write mode.
  - **Data Width:** 32-bit.
  - **Depth:** 4*4096 (16KB Total Capacity).
  - **Latency:** 1 clock cycle, perfectly matching the F1 -> F2 and M1 -> M2 pipeline stages.

UART

To verify the AXI-Lite Bridge logic designed in RTL, we connected it to a compliant AXI-Lite UART IP.

- **IP Used:** AXI Uartlite.
- **Interface:** AXI4-Lite Slave.

- **Functionality:** It bridges the high-speed processor bus to the slower UART serial lines. It includes internal TX/RX FIFOs to buffer data, preventing CPU stalls during high-speed transmission.
- **Address Map:** Mapped to base address 0x10000000.

Implementation Results

1. Resource Utilization

The design was synthesized targeting the xc7z020. The post-implementation utilization report indicates a compact design, leaving ample resources for future extensions.

Resource	Utilization	Available	Utilization %
LUT	2530	53200	4.76
LUTRAM	10	17400	0.06
FF	2364	106400	2.22
BRAM	4	140	2.86
IO	4	125	3.20
BUFG	2	32	6.25
MMCM	1	4	25.00

2. Timing Analysis

Timing closure is critical for pipelined processors. We performed Timing Analysis to verify setup and hold times.

- **Target Constraint:** 16.67ns (60 MHz)
- **Worst Negative Slack (WNS):** +0.697 ns
    - Positive WNS indicates that the design meets the frequency requirements.
- **Worst Hold Slack (WHS):** +0.117 ns
- **Max Estimated Frequency (F_{max}):**

$$F_{max} = \frac{1}{T_{target} - WNS} \approx 62.62 \text{ MHz}$$